



CENTRO REGIONAL
UNIVERSITARIO
CÓRDOBA IUA

FACULTAD DE INGENIERÍA
CENTRO REGIONAL UNIVERSITARIO IUA

Desarrollo de Herramientas de SW
TRABAJO PRÁCTICO

INFORME TRABAJO FINAL

Autores:

OLGUIN, Daniel David

Profesor:

ESCHOYEZ, Maximiliano A.

Introducción

El objetivo de este trabajo es implementar una herramienta capaz de procesar archivos en código fuente escritos en una versión reducida del lenguaje C, abarcando las etapas de análisis léxico, sintáctico y semántico de un compilador, así como también la generación de código intermedio (código de tres direcciones) y su optimización básica.

Consigna

Dado un archivo de entrada en C reducido, se debe:

1. Generar un reporte de errores en caso de existir.
2. Generar la tabla de símbolos de dicha entrada (sólo en caso de no presentar errores).
3. Generar la versión en código de tres direcciones del código fuente (sólo en caso de no presentar errores).

Para ello se debe construir un parser que tenga como mínimo la implementación de los siguientes puntos:

- Reconocimiento de bloques de código delimitados por llaves y controlar balance de apertura y cierre.
- Verificación de la estructura de las operaciones aritmético/lógicas y las variables o números afectadas.
- Verificación de la correcta utilización del punto y coma para la terminación de instrucciones.
- Balance de llaves, corchetes y paréntesis.
- Tabla de símbolos.
- Llamado a funciones de usuario.

Si la fase de verificación gramatical no ha encontrado errores, se debe proceder a:

1. Detectar variables y funciones declaradas pero no utilizadas y viceversa.
2. Generar la versión en código intermedio usando código de tres direcciones.
3. Optimizar (a un nivel básico) el código intermedio generado.

Desarrollo

Tecnologías usadas

- Python 3 como lenguaje de implementación.
- ANTLR4 para la definición de la gramática y la generación automática de `compiladorLexer.py` y `compiladorParser.py`.
- Git y Github para el control de versiones.

Componentes principales

Compilador.g4: Gramática

La gramática `compilador.g4` define formalmente la sintaxis de una versión reducida del lenguaje C y constituye la base del análisis léxico y sintáctico del compilador. Fue implementada en ANTLR4 para generar automáticamente el lexer y el parser en Python.

Incluye:

- Definición de tokens para palabras reservadas, operadores, literales e identificadores.
- Soporte para declaraciones, asignaciones, estructuras de control (`if`, `while`, `for`), funciones (prototipos y definiciones), llamadas y `return`.
- Manejo de bloques con alcance léxico.
- Expresiones aritmético-lógicas con precedencia correcta de operadores, implementada mediante reglas jerárquicas.

El diseño separa claramente los distintos niveles de la expresión (lógico, comparación, aritmético, unario y postfijo), garantizando un árbol sintáctico coherente y apto para el análisis semántico posterior.

EscuchaErroresSintácticos: Listener de errores sintácticos

Esta clase extiende el `ErrorListener` base de ANTLR4 para interceptar los errores detectados durante la fase de análisis sintáctico. Su objetivo es transformar los mensajes genéricos del parser en notificaciones más amigables y precisas para el usuario.

Funcionalidades clave:

- Refinamiento de Mensajes: Identifica patrones específicos en los errores de ANTLR (como `mismatched input` o `no viable alternative`) para diagnosticar fallos comunes:
 - Omisión de símbolos de cierre (`'`), `}` , `;`).
 - Errores en la apertura de paréntesis o llaves.
 - Formato incorrecto en listas de declaración de variables.
- Ajuste de Contexto Lineal: Implementa una lógica de retroceso en el flujo de tokens. Si el parser detecta un error al inicio de una línea (como la falta de un `;`), el listener busca el token anterior para reportar la línea exacta donde realmente se cometió la omisión.
- Gestión de Errores: Almacena los mensajes en una lista interna para su posterior reporte y, de ser necesario, bloquea el paso a la etapa de generación de código si se encuentran inconsistencias.

Escucha: Analizador semántico mediante Listener

El análisis semántico se implementa mediante un `Listener` de ANTLR4, lo que permite ejecutar acciones al entrar o salir de cada regla del árbol de parseo.

Principales responsabilidades:

- Construcción y gestión de la tabla de símbolos con soporte para contextos anidados.
- Control de declaraciones duplicadas y uso de identificadores no declarados.
- Verificación de variables sin inicializar.
- Validación de tipos en expresiones, asignaciones, llamadas a función y sentencias **return**.
- Control de consistencia entre prototipos y definiciones de funciones.
- Detección de funciones declaradas pero no definidas.

Los errores se registran con información contextual y, en caso de fallas semánticas, se impide la generación de código.

Se eligió el patrón Listener (y no Visitor) para esta fase porque:

- Permite validar progresivamente durante el recorrido.
- Facilita la construcción incremental de la Tabla de Símbolos.
- Se adapta naturalmente al manejo de contextos anidados.

Caminante: Generador de código intermedio mediante Visitor

Implementa la generación de código de tres direcciones utilizando el patrón Visitor de ANTLR4. A diferencia del análisis semántico, esta fase requiere un recorrido explícito del árbol para gestionar el flujo de datos y el retorno de valores intermedios.

Características y lógica de traducción:

- Gestión de símbolos: Utiliza temporales (tX) para resultados intermedios y etiquetas (LX) para el control de flujo.
- Expresiones: Traduce operaciones aritmético-lógicas respetando la precedencia y asociatividad del árbol sintáctico.
- Estructuras de control: Modela ciclos (while, for) y condicionales (if/else) mediante instrucciones de salto condicional (ifnot), saltos incondicionales (jmp) y puntos de entrada (label).
- Gestión de funciones: Implementa una convención basada en pila para el paso de parámetros, el manejo de la dirección de retorno y la recuperación del valor devuelto.

Optimizador

El módulo Optimizador aplica transformaciones sobre el código intermedio generado, trabajando de manera iterativa hasta alcanzar un punto fijo ("sin más cambios").

Estrategia general

- El código se procesa como texto.
- Se utilizan expresiones regulares para reconocer patrones de instrucciones.
- Las optimizaciones se aplican en ciclos sucesivos:
 1. Plegado de constantes: Evalúa en tiempo de compilación expresiones cuyos operandos son constantes.
 2. Propagación de copia: Realiza propagación lineal hacia adelante:
 - Sustituye variables por constantes conocidas.
 - Simplifica asignaciones encadenadas.
 - Aplica pequeñas optimizaciones locales (peephole).
 - Invalida el análisis ante instrucciones con efectos
 - No realiza análisis global de flujo de control.

3. Eliminación de código muerto: Elimina asignaciones a temporales que no vuelven a utilizarse. Se realiza en dos pasadas:
 - Conteo de usos de variables.
 - Eliminación de temporales sin usos posteriores.

Conclusión

La gramática desarrollada cubre de manera consistente una versión reducida del lenguaje C, suficiente para satisfacer los objetivos planteados en la consigna. No obstante, no contempla características más avanzadas del lenguaje, tales como **structs**, punteros, manejo de memoria dinámica o macros, lo cual limita el alcance del compilador implementado.

En cuanto a la optimización, el sistema incorpora transformaciones básicas —plegado de constantes, propagación lineal hacia adelante y eliminación de código muerto— que permiten mejorar la eficiencia del código intermedio generado. Sin embargo, no se implementaron optimizaciones globales más sofisticadas que podrían potenciar aún más la calidad del código resultante.

La verificación de efectos secundarios y llamadas con comportamientos complejos fue abordada de manera conservadora, priorizando la corrección semántica por sobre la agresividad en la optimización. Como consecuencia, en ciertos casos el sistema puede preservar instrucciones potencialmente redundantes para evitar transformaciones inseguras.

Asimismo, el mecanismo de detección de errores —incluyendo las heurísticas implementadas en el listener sintáctico personalizado— puede ampliarse para cubrir un mayor espectro de patrones y ofrecer mensajes aún más precisos o sugerencias de recuperación.

A pesar de estas limitaciones, se logró desarrollar una herramienta plenamente funcional, implementada en Python y basada en ANTLR4, capaz de realizar análisis léxico, sintáctico y semántico sobre una versión reducida de C, generar la tabla de símbolos correspondiente y producir código intermedio en forma de tres direcciones. De este modo, el sistema cumple con los requerimientos establecidos para el trabajo final. Además, su diseño modular —con Listeners para el análisis semántico, un Visitor para la generación de código y un optimizador desacoplado— favorece la mantenibilidad del proyecto y sienta una base sólida para futuras extensiones y mejoras.